Software Architecture and Design Document

Online Exam Portal

Team 6 | Version 1.0 | Draft for review

Field	Value
Project	Online Exam Portal
Document	Software Architecture and Design (SAD)
Team	Team 6
Version	1.0
Status	Draft for review
Related document	Online Exam Portal SRS v1.0

Team Members

Name	SRN
Bhumika Praveen Patil	PES1UG24AM352
Akash Shivanand Bagoji	PES1UG24AM339
VijaySuriyaP	PES1UG24AM320
Thejas	PES1UG24AM306

Document Control

Item: Details

Version: 1.0

Date: 19 October 2026

Status: Draft for review

Project: Online Exam Portal

Team: Team 6

Source baseline: Online Exam Portal SRS v1.0 dated 06-09-2026

Table of Contents

- 1. Introduction
- 2. Architectural Goals and Constraints
- 3. Architecture
- 4. Detailed Design
- 5. Architecture Decisions, Risks and Traceability
- Appendix A: Component Diagram
- Appendix B: Component Glossary

1. Introduction

1.1 Purpose

This Software Architecture and Design (SAD) document describes the architecture, component decomposition, interfaces, and detailed design of the Online Exam Portal developed by Team 6. It translates the requirements in the Online Exam Portal Software Requirements Specification (SRS), Version 1.0, into a technical blueprint for implementation, integration, testing, and maintenance.

1.2 Scope

The system is a web-based platform for multiple-choice examinations. Administrators manage student accounts, examinations, questions, schedules, and results. Students authenticate, view available examinations, take timed examinations, submit answers, and view evaluated results. The initial scope excludes descriptive-answer evaluation, online payment processing, and advanced live video proctoring.

1.3 Intended Audience

The intended audience includes the project team, faculty/evaluators, developers, testers, administrators, and future maintainers. The document supports implementation planning and verification that the design addresses the SRS.

1.4 Definitions and Abbreviations

OEP: Online Exam Portal; SRS: Software Requirements Specification; SAD: Software Architecture and Design; MCQ: Multiple-Choice Question; API: Application Programming Interface; ORM: Object-Relational Mapper; UI: User Interface; HTTPS: Hypertext Transfer Protocol Secure.

1.5 References

Online Exam Portal, Software Requirements Specification (SRS), Version 1.0, dated 06-09-2026; UML component-diagram conventions; course-provided Software Architecture and Design document structure.

1.6 Document Overview

Section 2 states the architectural goals and constraints. Section 3 describes the architecture, component view, interfaces, layers, and deployment. Section 4 describes important runtime flows, API design, validation, and error handling. Section 5 records decisions, risks, and traceability.

2. Architectural Goals and Constraints

2.1 Architectural Drivers

The architecture is driven by secure student and administrator authentication; role-based access control; administration of examinations and MCQ questions; timed attempts and answer submission; automatic evaluation and score calculation; persistent results; and responsive web access.

2.2 Quality Attributes

Security: protect credentials and examination data and enforce roles on the server. Performance: dashboards should load within 3 seconds and requested examination content within 2 seconds under normal load. Reliability: submitted answers and results should persist. Usability: provide clear navigation, readable controls, and meaningful status/error messages. Maintainability: separate UI, request handling, domain logic, and persistence.

2.3 Constraints

The SRS specifies HTML, CSS, and JavaScript for the frontend; Python with Django for the backend; SQLite during development, with PostgreSQL as a possible deployment database; standard web browsers; and a network connection. Initial examination type is MCQ. Advanced live video proctoring, descriptive-answer evaluation, and online payment are outside scope.

2.4 Assumptions

Users access the portal through a supported browser and a stable network. Student and administrator accounts are provisioned or registered according to the implemented registration policy. Examination availability, duration, questions, options, and correct answers are configured by authorized administrators. HTTPS is used in deployment.

3. Architecture

3.1 Architectural Style

The Online Exam Portal uses a web-based client-server architecture with a layered Django application. The presentation tier runs in a browser. The application tier contains Django request routing, authentication and authorization, examination domain services, evaluation, and result handling. The data tier stores user, examination, question, attempt, answer, and result information through Django ORM.

3.2 Architectural Principles

Separation of concerns: browser presentation, business logic, and persistence remain distinct. Server-side enforcement: role checks and score calculation are performed by trusted backend logic, not by client-side code alone. Single responsibility: authentication, exam management, attempts, evaluation, and results have distinct responsibilities. Data integrity: answers and results are associated with the correct student and examination. Secure defaults: reject unauthenticated or unauthorized requests.

3.3 Component View

The component diagram shows the browser and web interface communicating with the Django application. The Django application delegates requests to authentication, student management, exam/question management, attempt/answer submission, evaluation/score calculation, and result-management components. Components access persistent data through Django ORM to SQLite during development or PostgreSQL if adopted for deployment.

3.4 Interface View

The browser sends HTTPS requests in deployment and receives HTML pages or application responses. Django URL routing maps requests to views/handlers. Authentication checks credentials and session state; authorization checks whether the current role may perform the requested operation. Domain components use ORM models and queries to read or update stored records. Results are returned to the browser after successful processing.

3.5 Layered Decomposition

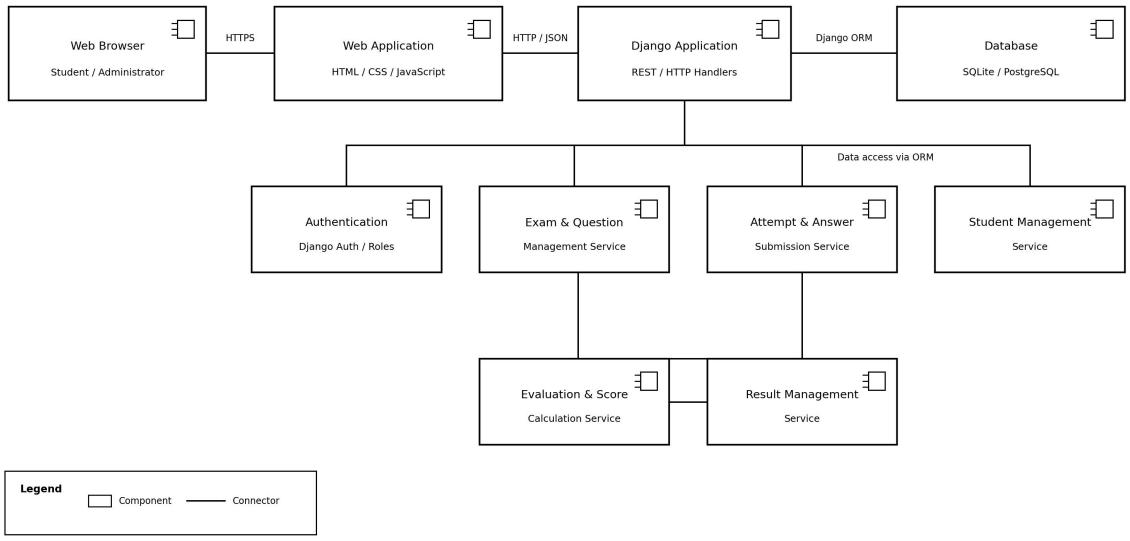
Presentation layer: login, student dashboard, admin dashboard, examination page, answer selection, timer display, submission confirmation, and result pages. Web/API layer: Django URL routing, request handling, forms/serializers where used, and response generation. Security layer: Django authentication, role-based access checks, and input validation. Domain layer: student management, exam/question management, examination attempts, answer submission, automatic evaluation, and result management. Data layer: Django ORM models and SQLite/PostgreSQL.

3.6 Deployment View

During development, a browser accesses the Django development server and the server accesses SQLite. In a deployment configuration, the Django application runs on a server and may use PostgreSQL. Browser-to-server communication should use HTTPS. No specialized client hardware is required; a modern browser and network connection are sufficient.

Figure 1 — Component diagram: logical decomposition of the Online Exam Portal.

SE Component Diagram – Online Exam Portal



4. Detailed Design

4.1 Design Overview

A typical request begins when a student or administrator selects an action in the browser. The browser sends a request to Django. The request handler checks authentication and role permissions, validates submitted fields, calls the relevant domain logic, and uses ORM operations to retrieve or persist data. A response is returned to the browser. For answer submission, the system records the attempt and answers, evaluates selected options against stored correct answers, calculates a score, persists the result, and displays it to the student.

4.2 Sequence Flow 1: Login and Role Resolution

1. The user opens the login page and enters credentials. 2. The browser submits the login request to Django. 3. Django validates the request and checks the credentials using its authentication mechanism. 4. Invalid credentials are rejected with an appropriate message. 5. On success, the authenticated user and role are resolved. 6. Access to student or administrator pages is controlled according to the role. 7. The browser displays the appropriate dashboard.

4.3 Sequence Flow 2: Taking and Submitting an Examination

1. The student signs in and opens the dashboard. 2. The portal lists examinations available during the permitted period. 3. The student starts an examination. 4. The examination page displays questions, MCQ options, navigation controls, and the configured timer. 5. The student selects or changes answers before submission. 6. The student submits the attempt, or the system submits it when time expires. 7. Django validates the attempt and records the submitted answers. 8. Evaluation logic compares selected options with the stored correct answers and calculates the score. 9. The result is stored and made available to the student; administrators may view results for completed examinations.

4.4 API / Request Design

The SRS describes browser-to-Django requests for login, examination data, answer submission, and result retrieval. The following are logical request groups, not a claim that these exact routes have already been implemented: Authentication (login/logout); Student management (registration, profile, administrative account management); Examination management (create, update, deactivate/delete, list available exams); Question management (add, edit, delete MCQs); Attempts (start attempt, fetch questions, save/select answers, submit attempt); Results (retrieve a student's result and administrator result listings). Use HTTP semantics consistently and protect privileged operations with server-side role checks.

4.5 Error Handling

Invalid credentials should be rejected without exposing sensitive details. Unauthenticated requests to protected pages should be redirected to login or rejected. Authenticated users without permission should receive an access-denied response/page. Invalid or incomplete form data should produce clear validation feedback. Requests for unavailable examinations or missing records should show a not-found or unavailable message. Database or unexpected server failures should be handled without exposing stack traces or secrets to end users.

4.6 UX and Validation Design

Provide clear labels, navigation, and meaningful status messages. Validate required fields and permitted values on the server even if browser-side validation is also used. Prevent students from accessing administrator functions and prevent students from modifying stored scores directly. The timer should reflect the configured examination duration, and expiry should trigger automatic submission as required by the SRS. Results should be associated with the relevant student and examination.

5. Architecture Decisions, Risks and Traceability

5.1 Architecture Decision Records

ADR-001 — Django backend: accepted because the SRS specifies Python with Django for authentication, request handling, examination management, evaluation, and results. ADR-002 — Browser-based frontend: accepted because the SRS specifies HTML, CSS, and JavaScript and access through standard browsers. ADR-003 — ORM-based persistence: accepted because the SRS identifies Django ORM as the database interface. ADR-004 — SQLite initially, PostgreSQL as an option: SQLite is the initial development database; PostgreSQL may be adopted if deployment needs require it. ADR-005 — Role checks on the server: accepted to protect administrative operations, correct answers, submitted answers, and results.

5.2 Risks and Mitigations

Incorrect or incomplete answer persistence could lose examination work; mitigate through careful transaction handling and submission tests. Timer/client clock differences could affect expiry; treat server-side state as authoritative where implemented. Weak role checks could expose administrative functions or correct answers; test unauthorized access explicitly. Poorly configured questions or answer keys could produce incorrect scores; validate question data and test evaluation logic. Database choice may change; keep persistence behind ORM models. Network interruption may disrupt a live attempt; provide clear failure feedback and test the supported recovery behaviour.

5.3 Requirements Traceability

OEP-F-001 to F-003 (authentication and role access): Authentication component and server-side authorization. OEP-F-004 to F-006 (student registration/profile/admin management): Student Management component. OEP-F-007 to F-010 (exam creation, updates, removal/availability): Exam Management component. OEP-F-011 to F-013 (MCQ question CRUD): Question Management component. OEP-F-014 to F-019 (start exam, show questions, timer, answer selection, submit, automatic expiry submission): Attempt and Answer Submission components. OEP-F-020 to F-023 (evaluate, calculate, store, display results): Evaluation and Result Management components. OEP-F-024 (administrator result viewing): Result Management with administrator authorization. OEP-NF-001 and OEP-NF-002 (response times): request handling, database queries, and page design. OEP-NF-003 (answer/result persistence): ORM and database. OEP-NF-005 (usability): browser UI. OEP-SR-001 to OEP-SR-005 and SO-OEP-001 to SO-OEP-003 (authentication, password protection, role access, protected exam/result data): Django authentication, authorization checks, validation, and controlled persistence.

5.4 Future Work

Potential future work, subject to scope approval, includes production deployment hardening, expanded integration and load testing, improved recovery for temporary network interruptions, audit logging for administrative actions, and evaluation of PostgreSQL for deployment. Advanced video proctoring, descriptive-answer evaluation, and online payments remain outside the initial SRS scope.

Appendix A: Component Diagram

SE Component Diagram – Online Exam Portal

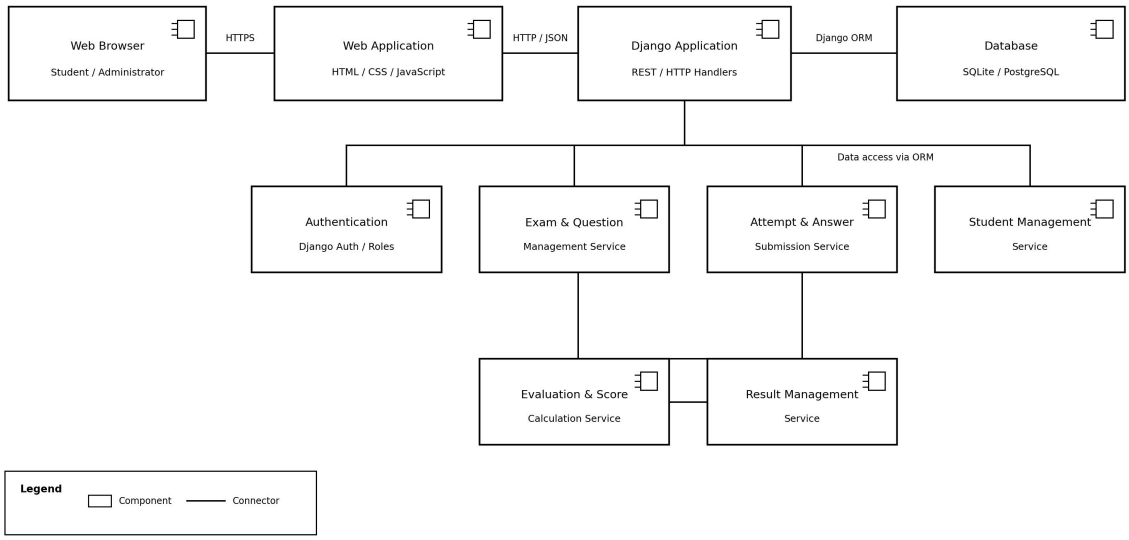


Figure A.1 — Online Exam Portal component diagram, Team 6.

Appendix B: Component Glossary

Component	Layer	Responsibility
Web Browser	Presentation	Provides student and administrator access to the portal.
Web Application	Presentation	Renders login, dashboards, exam pages, answer selection, and results using HTML/CSS/JavaScript.
Django Application	Web/Application	Routes requests, coordinates components, validates requests, and returns responses.
Authentication	Security / Domain	Verifies credentials and resolves authenticated user identity and role using Django authentication.
Student Management	Domain	Supports student registration/profile and administrator account management.
Exam & Question Management	Domain	Creates, updates, deactivates, schedules examinations and manages MCQ questions/options.
Attempt & Answer Submission	Domain	Starts attempts, records answer selections, and handles explicit or timer-triggered submission.
Evaluation & Score Calculation	Domain	Compares submitted choices with stored correct answers and calculates scores.
Result Management	Domain	Persists evaluated results and provides authorized result retrieval.
Django ORM	Persistence interface	Maps application objects to database operations.
SQLite / PostgreSQL	Data	Stores user, exam, question, attempt, answer, and result records.